

Produire un algorithme

Avant de commencer à programmer, il est important de bien comprendre la tâche à faire et le contexte d'utilisation. Souvent, il est possible d'utiliser plusieurs solutions différentes pour arriver à un résultat. L'objectif de cette étape est de choisir la meilleure solution possible en fonction des contraintes et des objectifs.

Voici quelques étapes pour produire un algorithme :

1. **Analyse du problème** : Comprendre les exigences et les contraintes du problème à résoudre. On décortique un problème complexe en sous-problèmes plus simples.
2. **Recherche de solutions** : Explorer différentes approches et techniques pour résoudre le problème. (organiser les idées avec des diagrammes, des listes, etc.)
3. **Conception de l'algorithme** : Décrire l'algorithme de manière formelle, en utilisant des étapes claires et logiques. (utiliser des pseudocodes, des diagrammes de flux, etc.)
4. **Validation de l'algorithme** : Tester l'algorithme avec des cas d'utilisation pour s'assurer qu'il fonctionne comme prévu. Essayez de voir les cas limites. Dans quel contexte est-ce que mon programme pourrait arrêter de fonctionner? Qu'est-ce que l'utilisateur pourrait faire pour provoquer une erreur dans mon programme?
5. **Écrire les étapes sous forme de pseudo-code en commentaire dans mon code** : Cela permet de garder une trace de la logique de l'algorithme et de faciliter sa compréhension avant de programmer.
6. **Implémentation** : Traduire l'algorithme en code dans le langage de programmation choisi.

En suivant ces étapes, vous serez en mesure de produire un algorithme efficace et adapté à votre problème.

Mots-clés lors de la rédaction du pseudo-code

Utilisez ces mots-clés pour structurer votre pseudo-code :

- **INITIALISER** : pour définir une variable et lui donner une valeur de départ.
- **AFFECTER** : pour donner une nouvelle valeur à une variable.
- **SI ... ALORS ... SINON SI ... SINON ... FIN SI** : pour les conditions.
- **TANT QUE ... FAIRE ... FIN TANT QUE** : pour les boucles conditionnelles.
- **POUR ... DE ... À ... FAIRE ... FIN POUR** : pour les boucles avec un nombre d'itérations connu.
- **AFFICHER** : pour afficher un message à l'utilisateur.
- **DEMANDER** : pour demander une entrée à l'utilisateur.
- **INCREMENTER** : pour augmenter la valeur d'une variable numérique.
- **DECREMENTER** : pour diminuer la valeur d'une variable numérique.
- **ATTENDRE** : pour indiquer qu'une action doit se produire avant de continuer (ex: attendre un clic, attendre le chargement de la page).
- **NomDeSousProgramme** : pour définir une fonction ou un sous-programme, donnez lui un nom clair et descriptif.
- **TERMINER NomDeSousProgramme** : pour indiquer la fin d'un sous-programme ou d'une fonction.

Exemple pour un jeu mini-jeu des devinettes

Contexte

Vous devez créer un mini-jeu de devinettes où le joueur doit répondre correctement à 3 questions sur un même sujet pour gagner. Le joueur a droit à 3 erreurs avant de perdre. Après chaque question, le joueur doit recevoir un message indiquant si sa réponse était correcte ou incorrecte, ainsi que le nombre de questions restantes et le nombre d'erreurs restantes. Le jeu n'a pas d'interface graphique. Le joueur interagit via les boîtes de dialogue du navigateur (prompt, alert, confirm).

Analyse du problème

- Le joueur doit répondre correctement à 3 questions pour gagner.
- Il a droit à 3 erreurs.
- Après chaque question, le joueur reçoit un retour sur sa réponse et des informations sur l'état du jeu (questions restantes, erreurs restantes).
- Lorsque les 3 questions ont été posées, le jeu se termine et le joueur reçoit un message indiquant s'il a gagné ou perdu.
- Le jeu n'a pas d'interface graphique, donc l'interaction se fait via des boîtes de dialogue.

Recherche de solutions

- Utiliser une boucle pour poser les questions au joueur. For? While? Boucle jeu avec setInterval?
- Stocker les réponses du joueur et les comparer aux réponses correctes.
- Mettre à jour le nombre de questions restantes et d'erreurs restantes après chaque réponse.
- Afficher un message de fin de jeu avec le résultat final.
- Utiliser des fonctions pour organiser le code et rendre le programme plus lisible.

Conception de l'algorithme

```
```text
INITIALISER le nombre de questions à 3
INITIALISER le nombre d'erreurs à 0
INITIALISER le nombre de réponses correctes à 0

INITIALISER la question 1
INITIALISER la question 2
INITIALISER la question 3

POUR i DE 1 À nbQuestions FAIRE
 DEMANDER la question i
 NORMALISER la réponse (mettre en minuscules, enlever les espaces)

 TANT QUE la réponse est vide OU nulle FAIRE
 AFFICHER "Veuillez répondre à la question."
 DEMANDER la question i
 FIN TANT QUE

 SI la réponse est correcte ALORS
 AFFICHER "Correct! Il vous reste (5 - i) questions et (3 - erreurs)
erreurs."
 INCREMENTER le nombre de réponses correctes
 SINON
```

```

 AFFICHER "Incorrect! Il vous reste (5 - i) questions et (3 - erreurs)
erreurs."
 INCREMENTER le nombre d'erreurs
 FIN SI
FIN POUR

AFFICHER "Jeu terminé! Vous avez eu (nBonnesReponses) bonnes réponses et
(nErreurs) erreurs."

```

## Validation de l'algorithme

- L'utilisateur pourrait ne rien répondre (appuyer sur annuler) ou mettre une réponse vide. Dans ce cas, on repose la question.
- L'utilisateur pourrait répondre avec des majuscules ou des espaces. On doit normaliser la réponse avant de la comparer. (On peut utiliser la méthode `toLowerCase()` pour ignorer la casse et `trim()` pour enlever les espaces superflus.)
- L'utilisateur pourrait utiliser des caractères spéciaux ou des accents. On doit également normaliser la réponse pour gérer ces cas.
- On doit vérifier que le jeu se termine correctement lorsque le joueur gagne ou perd.

## Implémentation

```

// Définir les questions et les réponses
const sQuestion1 = "Quelle est la capitale de la France ?";
const sQuestion2 = "Quel est le plus grand océan du monde ?";
const sQuestion3 = "Quel est le plus grand lac du monde ?";

const sReponse1 = "paris";
const sReponse2 = "pacifique";
const sReponse3 = "caspienne";

// Initialiser les variables du jeu
let nErreurs = 0;
let nNbQuestions = 3;
let nBonnesReponses = 0;

// Pour chaque question, on la pose à l'utilisateur
// On utilise une boucle POUR car on connaît le nombre de questions à poser
for (let i = 0; i < nNbQuestions; i++) {
 let sQuestionUtilisateur = "";
 let sReponseAttendue = "";
 let sReponseUtilisateur = "";

 //Déterminer la question et la réponse attendue en fonction de l'itération de
 la boucle
 if (i == 0) {
 sQuestionUtilisateur = sQuestion1;
 sReponseAttendue = sReponse1;
 } else if (i == 1) {
 sQuestionUtilisateur = sQuestion2;
 sReponseAttendue = sReponse2;
 }
}

```

```

 } else if (i == 2) {
 sQuestionUtilisateur = sQuestion3;
 sReponseAttendue = sReponse3;
 }

 // Demander la réponse à l'utilisateur et la normaliser pour la comparer
 sReponseUtilisateur = prompt(sQuestionUtilisateur);
 sReponseUtilisateur = sReponseUtilisateur.toLowerCase().trim();

 // Tant que la réponse est vide ou nulle, on repose la question
 while (sReponseUtilisateur === "") {
 alert("Veuillez répondre à la question.");
 sReponseUtilisateur = prompt(sQuestionUtilisateur);
 }

 // Vérifier la réponse et mettre à jour les compteurs
 if (sReponseUtilisateur === sReponseAttendue) {
 alert("Correct!");
 nBonnesReponses = nBonnesReponses + 1;
 } else {
 alert("Incorrect!");
 nErreurs = nErreurs + 1;
 }

 // Afficher le nombre de questions et de bonnes réponses
 let nNombreQuestionsRestantes = nNbQuestions - i - 1;
 alert(
 `Jeu terminé! Vous avez ${nNombreQuestionsRestantes} questions restantes
 et ${nBonnesReponses} bonnes réponses.`
);
}

// Afficher le message de fin de jeu
alert(`Jeu terminé! Vous avez eu ${nBonnesReponses} bonnes réponses et ${nErreurs}
erreurs.`);

```

## Exemple pour le jeu d'attaque Pokemon avec clic de l'utilisateur

### Contexte

Vous devez créer un mini-jeu d'attaque Pokemon où le joueur peut cliquer sur le canvas pour attaquer. Le jeu doit gérer différents états: le tour du joueur et le tour de l'ordinateur. Le joueur doit cliquer pour attaquer pendant son tour, et l'ordinateur attaque automatiquement pendant son tour. Le jeu doit afficher des messages pour indiquer l'état actuel du jeu et les actions effectuées. Tant que le joueur ou l'ordinateur a des points de vie, le jeu continue. Si un des deux atteint 0 points de vie, le jeu se termine et un message indique qui a gagné. Le jeu possède une interface graphique qui affiche l'image du pokemon de l'utilisateur et celle de l'ordinateur ainsi qu'une zone pour les points de vies et les messages d'état.

### Analyse du problème

- Le jeu doit gérer deux états: le tour du joueur et le tour de l'ordinateur.
- Le joueur doit cliquer pour attaquer pendant son tour.

- L'ordinateur doit attaquer automatiquement pendant son tour.
- Le jeu doit afficher des messages pour indiquer l'état actuel du jeu et les actions effectuées.
- Tant que le joueur ou l'ordinateur a des points de vie, le jeu continue.
- Si un des deux atteint 0 points de vie, le jeu se termine et un message indique qui a gagné.
- Le jeu doit avoir une interface graphique pour afficher les éléments visuels. Canvas? Images HTML?

## Recherche de solutions

- Utiliser une variable pour suivre l'état actuel du jeu (tour du joueur ou tour de l'ordinateur).
- Utiliser des événements de clic pour permettre au joueur d'attaquer.
- Utiliser une boucle de jeu pour gérer les mises à jour et le rendu graphique avec setInterval
- Mettre à jour les points de vie et vérifier les conditions de fin de jeu après chaque attaque.
- Afficher les messages d'état et les points de vie sur le canvas.
- Utiliser des fonctions pour organiser le code et rendre le programme plus lisible.

## Conception de l'algorithme

```

INITIALISER les points de vie du joueur et de l'ordinateur
INITIALISER l'état du jeu à "tourJoueur"
INITIALISER les images des pokemons et les afficher sur le canvas

ATTENDRE chargement de la page
 DEMARRER BoucleJeu avec setInterval
FIN ATTENDRE

BOUCLEJEU
 INITIALISER nDegats;

 SI etat == "tourJoueur" ALORS
 ATTENDRE clic de l'utilisateur
 AFFECTER nDegats une valeur aléatoire entre 10 et 30
 RETIRER nDegats aux points de vie de l'ordinateur
 METTRE À JOUR l'affichage des points de vie
 CHANGER l'état du jeu à "tourOrdinateur"
 SINON SI etat == "tourOrdinateur" ALORS
 AFFECTER nDegats une valeur aléatoire entre 10 et 30
 RETIRER nDegats aux points de vie du joueur
 METTRE À JOUR l'affichage des points de vie
 CHANGER l'état du jeu à "tourJoueur"
 FIN SI

 SI points de vie du joueur <= 0 OU points de vie de l'ordinateur <= 0
ALORS
 CHANGER l'état du jeu à "finJeu"
 DEMARRER TerminerJeu
 FIN SI
FIN BOUCLEJEU

TERMINERJEU
 AFFICHER "Vous avez gagné!"

```

## Implementation

```
//Définir les variables du jeu
let nPointsVieJoueur = 100;
let nPointsVieOrdinateur = 100;
let etatJeu = "tourJoueur"; // "tourJoueur", "tourOrdinateur", "finJeu"

// Initialiser le canvas et le contexte
const oCanvasHTML = document.querySelector("#monCanvas");
const oContexte = oCanvasHTML.getContext("2d");

// Initialiser les images des pokemons
const oImagePokemonJoueur = new Image();
oImagePokemonJoueur.src = "pokemon_joueur.png"; // Remplacer par le chemin

const oImagePokemonOrdinateur = new Image();
oImagePokemonOrdinateur.src = "pokemon_ordinateur.png"; // Remplacer par le chemin

/**
 * Initialiser le jeu
 */
function initialiserJeu() {
 oInterval = setInterval(boucleJeu, 1000 / 60); // 60 FPS
 oCanvasHTML.addEventListener("click", clicJoueur);
}

function clicJoueur(event) {
 //Si c'est le tour du joueur, il peut cliquer pour attaquer
 if (etatJeu === "tourJoueur") {
 let nDegats = Math.floor(Math.random() * 21) + 10; // Dégâts aléatoires
 // entre 10 et 30
 nPointsVieOrdinateur -= nDegats;

 if (nPointsVieOrdinateur < 0) {
 nPointsVieOrdinateur = 0;
 }

 if (nPointsVieOrdinateur < 0) {
 nPointsVieOrdinateur = 0;
 }
 etatJeu = "tourOrdinateur";
 }
}

/**
 * Fonction appelée à chaque frame (60 fois par seconde)
 */
function boucleJeu() {
 oContexte.clearRect(0, 0, oCanvas.width, oCanvas.height);
```

```

// Afficher les points de vie
if (etatJeu === "finJeu") {
 clearInterval(oInterval);

 if (nPointsVieJoueur <= 0) {
 alert("Vous avez perdu!");
 } else {
 alert("Vous avez gagné!");
 }
} else if (etatJeu === "tourOrdinateur") {
 let nDegats = Math.floor(Math.random() * 21) + 10; // Dégâts aléatoires
entre 10 et 30
 nPointsVieJoueur -= nDegats;

 if (nPointsVieJoueur < 0) {
 nPointsVieJoueur = 0;
 }
 if (nPointsVieJoueur == 0 || nPointsVieOrdinateur == 0) {
 etatJeu = "finJeu";
 } else {
 etatJeu = "tourJoueur";
 }
} else if (etatJeu === "tourJoueur") {
 // Attendre le clic du joueur
 oContexte.fillText("C'est votre tour! Cliquez pour attaquer.", 50, 50);
}

// Afficher les images des pokemons
oContexte.drawImage(oImagePokemonJoueur, 50, 100, 100, 100);
oContexte.drawImage(oImagePokemonOrdinateur, 350, 100, 100, 100);
}
window.addEventListener("load", initialiserJeu);

```